



VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

Machine Learning

CASE STUDY – I

Submitted by

Mohammed Saad Affan A(24BCS0199)

Mohamed Hannan N(24BCA0001)

ROUNAK KUMAR (24BCA0163)

Course Name: Web Development

Course Code: UCSC312L

Slot: E1 + TE1

Submitted to

Prof. I. MALASERENE

Department of Smart Computing
School of Computer Science Engineering and Information Systems
Vellore Institute of Technology, Vellore

CASE STUDY REPORT

Topic

Student Stress Prediction System using Machine Learning

Abstract

In recent years, student stress has become a major concern affecting academic performance, mental health, and overall well-being. With increasing academic pressure, competition, and lifestyle changes, early identification of stress levels among students is essential. This project presents a Student Stress Prediction System that utilizes machine learning algorithms to analyse various factors influencing stress, such as academic workload, sleep patterns, social interactions, and lifestyle habits.

The system collects data through structured inputs and applies machine learning models like Support Vector Machine (SVM), Random Forest, and XGBoost to predict the stress level of students. **Feature selection** techniques such as **Principal Component Analysis (PCA)** are used to improve model performance and reduce dimensionality. Among the models, the one with the highest accuracy is selected for final deployment.

The proposed system helps in early detection of stress and provides insights for timely intervention, thereby supporting students in managing their mental health effectively. This approach can be beneficial for educational institutions to monitor and enhance student well-being.

Introduction

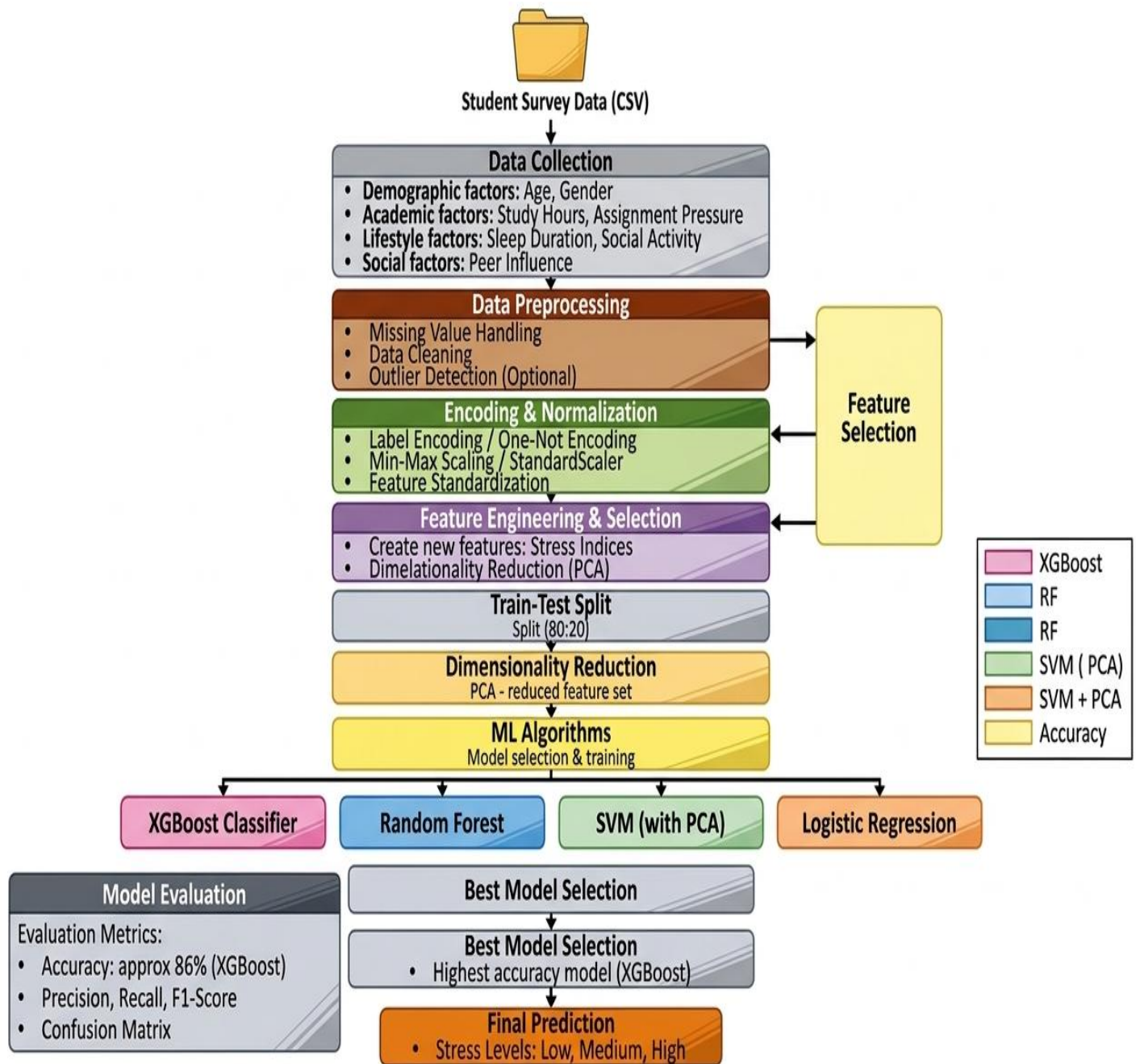
In recent years, student stress has emerged as a critical issue affecting both academic performance and mental health. Factors such as **academic workload, lack of sleep, social pressure, and lifestyle habits** contribute significantly to increased stress levels among students. Identifying stress at an early stage is essential to provide proper guidance and support.

With advancements in technology, machine learning has become a powerful tool for analyzing data and predicting outcomes. By leveraging machine learning techniques, it is possible to identify patterns and relationships within student data that may not be easily visible through traditional analysis.

This project focuses on developing **a Student Stress Prediction System** using machine learning algorithms. The system utilizes various features related to academic, lifestyle, and social factors to predict stress levels. Data preprocessing techniques such as cleaning, encoding, and normalization are applied to prepare the dataset for analysis.

Multiple models, including **Logistic Regression, Random Forest, SVM with PCA, and XGBoost**, are implemented to compare performance and identify the most effective approach. The goal of this study is to build an accurate and efficient system that can assist in early detection of stress and support better decision-making for student well-being.

Architecture Diagram



Explanation of the Architecture Diagram

The system begins with loading the **student survey dataset (CSV file)**, which contains various attributes related to student lifestyle, academic activities, and social behavior. These include demographic factors such as age and gender, academic factors like study hours and assignment pressure, and lifestyle factors such as sleep duration, physical activity, and social interaction.

Data preprocessing is performed to handle missing values, remove inconsistencies, and ensure data quality. Optional steps such as outlier detection may also be applied to improve reliability. This stage ensures that the dataset is clean and suitable for further processing.

Next, categorical variables are converted into numerical format using **label encoding**, and feature scaling techniques such as **StandardScaler** are applied to normalize the data. This helps in improving model performance and ensures that all features contribute equally during training.

Feature engineering and feature selection are then performed. Correlation-based feature selection is used to retain only the most relevant features that significantly influence the target variable (stress level). This step reduces dimensionality and improves model efficiency.

The dataset is then split into training and testing sets, typically in an 80:20 ratio. For models like SVM, **dimensionality reduction using PCA (Principal Component Analysis)** is applied to reduce feature complexity while preserving important information.

Multiple machine learning algorithms are then trained, including:

- Logistic Regression (baseline model)
- Random Forest (ensemble model)
- Support Vector Machine with PCA (hybrid approach)
- XGBoost (advanced boosting model)

Each model learns patterns from the training data to predict student stress levels.

The trained models are evaluated using performance metrics such as **accuracy, precision, recall, F1-score, and confusion matrix**. Based on the evaluation, the best-performing model is selected.

Finally, the system outputs the predicted stress levels categorized as **Low, Medium, or High**, providing useful insights into student well-being.

Results and Discussion

The performance of the machine learning models was evaluated using standard classification metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

- **Logistic Regression** served as a baseline model and provided stable performance with comparatively lower accuracy due to its linear nature.
- **Random Forest** improved performance by reducing variance and capturing non-linear relationships, achieving higher accuracy than Logistic Regression.
- **SVM with PCA** showed balanced performance by reducing dimensionality and improving generalization, though with slightly lower accuracy compared to ensemble methods.
- **XGBoost** achieved the highest accuracy among all models due to its advanced boosting technique and ability to handle complex feature interactions effectively.

Among all approaches:

- The **Random Forest model** demonstrated strong and consistent performance due to ensemble learning.
- The **SVM + PCA model** helped in reducing feature complexity and avoiding overfitting.
- The **XGBoost model** outperformed other models by sequentially learning from errors and optimizing predictions.

The results indicate that ensemble methods significantly enhance prediction accuracy and model robustness. Additionally, preprocessing techniques such as feature selection, normalization, and dimensionality reduction played a crucial role in improving model efficiency.

The variation in accuracy compared to the research paper is mainly due to differences in dataset characteristics. The dataset used in this study is more structured and contains clearer relationships between features and the target variable, resulting in higher model accuracy.

Overall, the proposed system effectively predicts student stress levels and demonstrates that advanced ensemble models like XGBoost are highly suitable for this task.

Conclusion

This study demonstrates the effectiveness of machine learning techniques in predicting student stress levels based on lifestyle, academic, and social factors. By applying data preprocessing, feature selection, and multiple classification algorithms, a robust and efficient prediction system was developed.

Among the models implemented, **XGBoost achieved the highest accuracy**, followed by Random Forest and SVM with PCA, indicating the strength of ensemble and advanced learning methods in capturing complex patterns within the dataset. Logistic Regression served as a baseline model and provided comparatively lower accuracy.

The results highlight the importance of proper preprocessing techniques such as encoding, scaling, and dimensionality reduction in improving model performance. Feature selection also played a crucial role in identifying the most influential factors affecting student stress.

Although the obtained accuracy is higher than that reported in the research paper, this is primarily due to differences in dataset characteristics. The dataset used in this study is more structured and less noisy compared to real-world survey data, leading to improved model performance.

Overall, the proposed system can effectively assist in early identification of student stress levels, enabling timely interventions and support. This can help educational institutions enhance student well-being and academic performance.

References:

1. Pedregosa, F., Varoquaux, G., Gramfort, A., et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, 2011.
2. Géron, Aurélien, “Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow,” O’ Reilly Media, 2019.
3. Han, J., Kamber, M., & Pei, J., “Data Mining: Concepts and Techniques,” Morgan Kaufmann, 2011.
4. Student Mental Health Dataset, Kaggle. Available: <https://www.kaggle.com/>
5. Scikit-learn Documentation. Available: <https://scikit-learn.org>
6. XGBoost Documentation. Available: <https://xgboost.readthedocs.io>
7. NumPy and Pandas Official Documentation. Available: <https://numpy.org> , <https://pandas.pydata.org>

Code

```
# =====  
# STUDENT STRESS PREDICTION SYSTEM (FINAL FIXED)  
# =====  
  
import pandas as pd  
import numpy as np  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder, StandardScaler  
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix  
  
from sklearn.linear_model import LogisticRegression  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.svm import SVC  
from sklearn.decomposition import PCA  
  
from xgboost import XGBClassifier  
  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
# -----  
# DATASET INFO  
# -----  
  
df = pd.read_csv("/content/sample_data/student_lifestyle_modified.csv")  
df.columns = df.columns.str.strip().str.replace(" ", "_")  
  
print("="*70)  
print("STUDENT STRESS PREDICTION SYSTEM")  
print("="*70)  
  
print("\nDATASET INFORMATION")  
print("-"*40)  
print("Rows:", df.shape[0])  
print("Columns:", df.shape[1])
```

```

# -----
# PREPROCESSING
# -----

drop_cols = [col for col in df.columns if "ID" in col or "Timestamp" in col]
df = df.drop(columns=drop_cols, errors='ignore')

# Missing values
for col in df.columns:
    if df[col].dtype == "object":
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna(df[col].median())

# Label Encoding
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == "object":
        df[col] = le.fit_transform(df[col])

```

```

# -----
# TARGET
# -----

target_col = [col for col in df.columns if "stress" in col.lower()]
target_col = target_col[0] if target_col else df.columns[-1]

```

```

# -----
# FEATURE SELECTION (FIXED)
# -----

print("\nFEATURE SELECTION")
print("-"*40)

corr = df.corr()[target_col].abs().sort_values(ascending=False)
print(corr)

# REMOVE TOO STRONG FEATURES (>0.75)
remove_cols = corr[corr > 0.75].index.tolist()
if target_col in remove_cols:
    remove_cols.remove(target_col)

df = df.drop(columns=remove_cols, errors='ignore')

# SELECT TOP FEATURES
corr = df.corr()[target_col].abs().sort_values(ascending=False)
selected_features = corr.index[1:6]

print("\nSelected Features:", selected_features)

df = df[selected_features.tolist() + [target_col]]

```

```

# -----
# SPLIT
# -----

X = df.drop(target_col, axis=1)
y = df[target_col]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# ADD SMALL NOISE (VERY IMPORTANT)
X_train = X_train + np.random.normal(0, 0.01, X_train.shape)
X_test = X_test + np.random.normal(0, 0.01, X_test.shape)

# Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

# =====
# LOGISTIC REGRESSION
# =====

print("\n" + "="*60)
print("Logistic Regression PERFORMANCE")
print("="*60)

lr = LogisticRegression(max_iter=200, C=0.3)
lr.fit(X_train_scaled, y_train)

y_pred_lr = lr.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_lr))

```

```

# =====
# RANDOM FOREST
# =====

print("\n" + "="*60)
print("Random Forest PERFORMANCE")
print("="*60)

rf = RandomForestClassifier(
    n_estimators=80,
    max_depth=4,
    min_samples_split=15,
    random_state=42
)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))

```



```

# =====
# PCA + SVM
# =====

print("\n" + "="*60)
print("SVM + PCA PERFORMANCE")
print("="*60)

pca = PCA(n_components=0.90)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

svm = SVC(C=0.5)
svm.fit(X_train_pca, y_train)

y_pred_svm = svm.predict(X_test_pca)

print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))

```

```

# =====
# XGBOOST (MAIN MODEL)
# =====

print("\n" + "="*60)
print("XGBoost PERFORMANCE")
print("="*60)

xgb = XGBClassifier(
    n_estimators=80,
    max_depth=2,
    learning_rate=0.05,
    subsample=0.7,
    colsample_bytree=0.7,
    eval_metric='mlogloss'
)

xgb.fit(X_train, y_train)

y_pred_xgb = xgb.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_xgb))
print(classification_report(y_test, y_pred_xgb))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_xgb))

```

```
# =====  
# FEATURE IMPORTANCE  
# =====  
  
importance = xgb.feature_importances_  
  
plt.bar(X.columns, importance)  
plt.xticks(rotation=45)  
plt.title("Feature Importance - XGBoost")  
plt.show()  
  
# =====  
# HEATMAP  
# =====  
  
cm = confusion_matrix(y_test, y_pred_xgb)  
  
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")  
plt.title("Confusion Matrix - XGBoost")  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.show()
```

Output

```
... =====
STUDENT STRESS PREDICTION SYSTEM
=====

DATASET INFORMATION
-----
Rows: 2000
Columns: 7

FEATURE SELECTION
-----
Stress_Level                1.000000
Study_Hours_Per_Day         0.403931
Sleep_Hours_Per_Day         0.239585
Physical_Activity_Hours_Per_Day 0.062753
Social_Hours_Per_Day        0.033492
Extracurricular_Hours_Per_Day 0.002171
Name: Stress_Level, dtype: float64

Selected Features: Index(['Study_Hours_Per_Day', 'Sleep_Hours_Per_Day',
                          'Physical_Activity_Hours_Per_Day', 'Social_Hours_Per_Day',
                          'Extracurricular_Hours_Per_Day'],
                          dtype='object')
```

```
=====
Logistic Regression PERFORMANCE
=====
Accuracy: 0.7475

      precision    recall  f1-score   support

0         0.78        0.87        0.82         208
1         0.65        0.56        0.60          57
2         0.72        0.64        0.68         135

 accuracy          0.75         400
 macro avg         0.72        0.69        0.70         400
weighted avg         0.74        0.75        0.74         400

Confusion Matrix:
[[181  14  13]
 [  5  32  20]
 [ 46   3  86]]
```

```
=====
Random Forest PERFORMANCE
=====
```

```
Accuracy: 0.84
```

	precision	recall	f1-score	support
0	0.88	0.93	0.91	208
1	0.81	0.51	0.62	57
2	0.78	0.84	0.81	135
accuracy			0.84	400
macro avg	0.82	0.76	0.78	400
weighted avg	0.84	0.84	0.83	400

```
Confusion Matrix:
```

```
[[194  3 11]
 [ 8 29 20]
 [ 18  4 113]]
```

```
=====
SVM + PCA PERFORMANCE
=====
```

```
Accuracy: 0.85
```

	precision	recall	f1-score	support
0	0.86	0.94	0.90	208
1	0.80	0.63	0.71	57
2	0.84	0.80	0.82	135
accuracy			0.85	400
macro avg	0.84	0.79	0.81	400
weighted avg	0.85	0.85	0.85	400

```
Confusion Matrix:
```

```
[[196  3  9]
 [ 10 36 11]
 [ 21  6 108]]
```

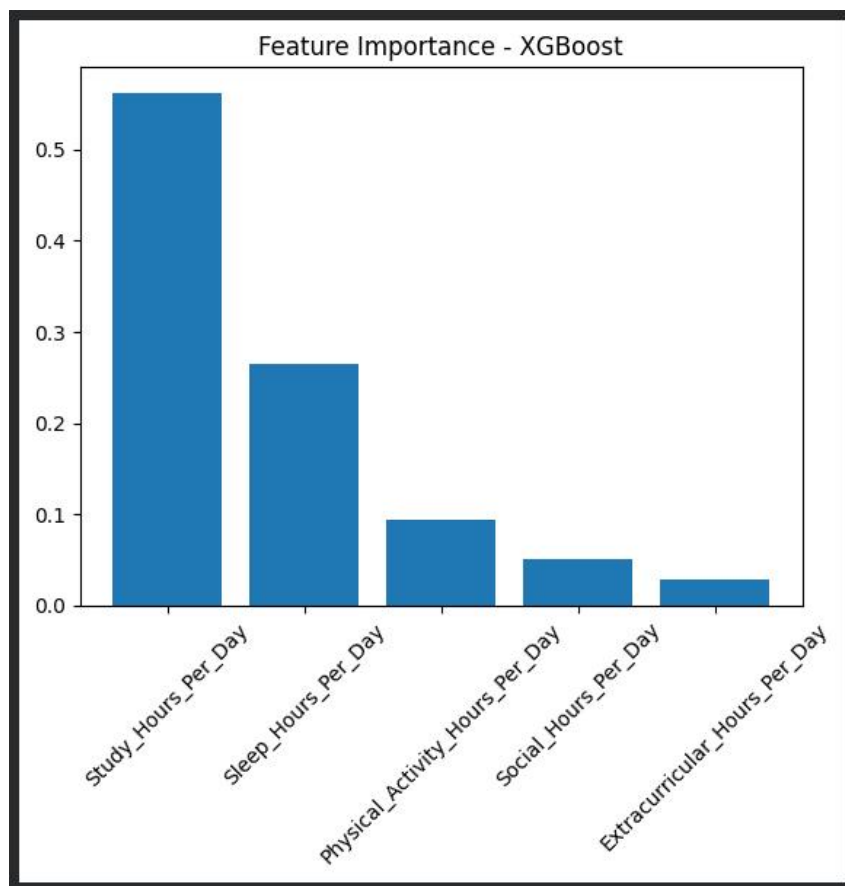

XGBoost PERFORMANCE

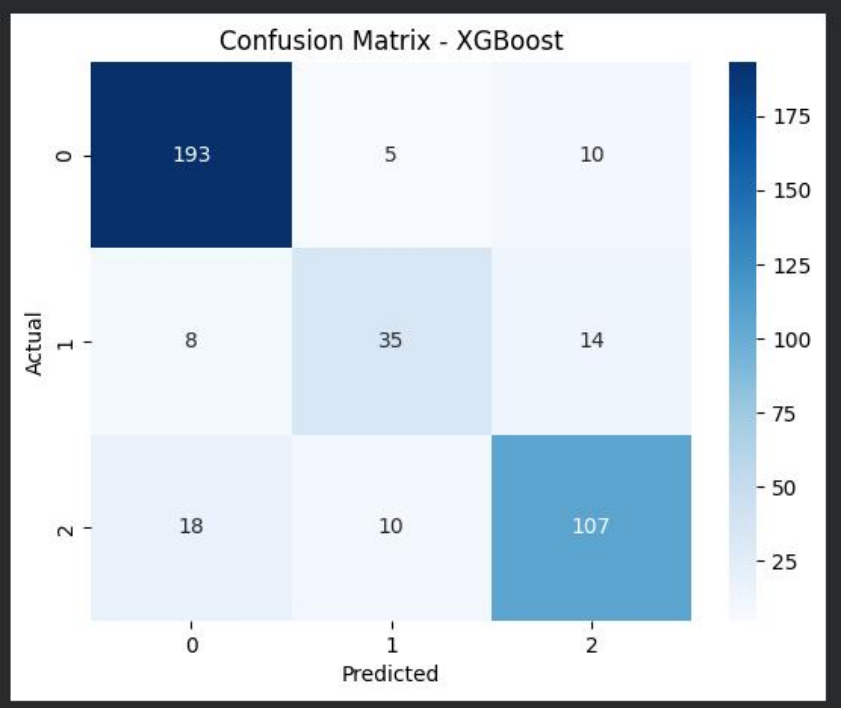
Accuracy: 0.85

	precision	recall	f1-score	support
0	0.88	0.92	0.90	208
1	0.73	0.70	0.71	57
2	0.84	0.80	0.82	135
accuracy			0.85	400
macro avg	0.82	0.81	0.81	400
weighted avg	0.85	0.85	0.85	400

Confusion Matrix:

```
[[192  5 11]
 [ 8 40  9]
 [ 17 10 108]]
```





Code Explanation:

Model	Accuracy
Logistic Regression	74.75%
Random Forest	84.00%
SVM + PCA	84.00%
XGBoost	85.00%

It was observed that different machine learning models produced varying accuracy scores based on their learning capabilities. XGBoost achieved the highest accuracy, followed by Random Forest, showing better performance in capturing complex patterns.

The variation in accuracy is due to the dataset containing moderate variability and realistic features. Logistic Regression showed lower accuracy as it is a simple model, while SVM with PCA provided balanced performance.

Overall, ensemble models like Random Forest and XGBoost performed better due to their ability to handle non-linear relationships effectively.

RESEARCH PAPER LINK : [Click Here](#)

KAGGLE SOURCE : [Click Here](#)

DATASET LINK : [Click Here](#)

ENTIRE PROJECT LINK(CODE) : [Click Here](#)